Department of Computer Engineering

CENG350 Software Engineering

Software Architecture Description (SAD) for KSO

Group #81

By

Alperen Ovak - 2580801

Mithat Can Timurcan - 2581064

Wednesday 14th May, 2025

Contents

List of Figures iii

List of Tables iv

1 Introduction 1

1.1 Purpose and objectives of KSO 1

1.2 Scope 2

1.3 Stakeholders and their concerns 2

2 References 4

3 Glossary 5

4 Architectural Views 6

4.1 Functional View 6

4.1.1 Stakeholders' Uses of This View 6

4.1.2 Component Diagram 8

4.1.3 Internal Interfaces 11

4.1.4 Interaction Patterns 13

4.1.5 External Interfaces 16

4.1.6 Interaction scenarios 18

4.2 Information View 20

4.2.1 Stakeholders' Uses of This View 20

4.2.2 Database Class Diagram 21

4.2.3 Operations on Data 22
4.3 Development View 25
4.3.1 Stakeholders' Uses of This View 25
4.3.2 Package Diagram 27
4.4 Deployment View 28
4.4.1 Stakeholders' Uses of This View 28
4.4.2 Deployment Diagram 30
4.5 Design Rationale 32

5 Architectural Views for Our Suggestions to Improve the Existing System 33

5.1 Functional View 33
5.1.1 Stakeholders' Uses of This View 33
5.1.2 Component Diagram 35
5.1.3 Internal Interfaces 37
5.1.4 Interaction Patterns 39
5.1.5 External Interfaces 40
5.1.6 Interaction scenarios 42
5.2 Information View 42
5.2.1 Stakeholders' Uses of This View 42
5.2.2 Database Class Diagram 44
5.2.3 Operations on Data 46
5.3 Development View 47
5.3.1 Stakeholders' Uses of This View 47
5.3.2 Package Diagram 49
5.4 Deployment View 50
5.4.1 Stakeholders' Uses of This View 50
5.4.2 Deployment Diagram 51
5.5 Design Rationale 52

List of Figures

4.1 Component Diagram of KSO 8
4.2 Internal Interfaces Class Diagram 11
4.3 Sequence Diagram for Annotate Footage 13
4.4 Sequence Diagram for Fine Tuning ML Models 14
4.5 Sequence Diagram for Analytics and Report Generation 15
4.6 External Interfaces Class Diagram 16
4.7 Activity Diagram for Annotate Footage 18
4.8 Activity Diagram for Fine Tuning ML Models 19
4.9 Database Class Diagram 21
4.10 Package Diagram 27
4.11 Deployment Diagram 30

5.1 Improved Component Diagram 35
5.2 Improved Internal Interfaces Class Diagram 37
5.3 Suggested Sequence Diagram for Video Classification 39
5.4 Improved External Interfaces Class Diagram 40
5.5 Suggested Activity Diagram for Annotate Footage 42
5.6 Improved Database Class Diagram 44
5.7 Improved Package Diagram 49
5.8 Improved Deployment Diagram 51

List of Tables

1 Revision History v

1.1 Stakeholders and their primary concerns, mapped to SAD views. 3

4.1 Operations on Data 22

5.1 Extended Operations on Data 46

Revision History

Date	Reason For Changes	Version
09.05.2025	First Draft	1.0
14.05.2025	Final Draft	2.0

Table 1: Revision History

1. Introduction

This Software Architecture Description (SAD) for the Koster Seafloor Observatory (KSO) is written in accordance with ISO/IEC/IEEE 42010:2011 [1] and adopts viewpoints from Rozanski & Woods' *A Viewpoint Catalog* [2] to structure four principal views—Functional, Information, Development, and Deployment—each addressing specific stakeholder concerns and quality-attribute trade-offs.

1.1 Purpose and objectives of KSO

The KSO system integrates video data management, citizen-science annotation, and machine-learning inference to accelerate marine-biodiversity research. Its primary objectives are to:

- Ingest and archive ROV/AUV subsea movies in a scalable hot/cold storage architecture.
- Enable citizen scientists to classify clips and draw bounding-box annotations via an intuitive web interface, aggregating results through consensus algorithms.
- Train and serve deep-learning models (e.g., YOLOv3) for automated object detection, exposing them through a REST API and interactive dashboard. [3]
- Visualize and export ecological metrics—species counts, distributions, and trends—to standard formats for downstream analysis.
- Maintain openness and extensibility, providing modular code, clear documentation, and integration points for new workflows or models.

1.2 Scope

In scope

- **Data Management Module**: Video segmentation, metadata extraction (Darwin Core–compliant), SQLite/PostgreSQL storage, backup and archival pipelines.
- **Citizen-Science Module**: Two Zooniverse workflows (clip classification, frame annotation), tutorial management, aggregation thresholds, and export utilities.
- **Machine-Learning Module**: Preprocessing (background correction, augmentation), model training/validation on GPU cluster, hyperparameter tuning, RESTful inference service, and Streamlit-based visualization.
- **User Interfaces**: Web portal for annotation, dashboard for model performance metrics and data exports.

Out of scope

- ROV/AUV mission planning or vehicle control.
- Expert taxonomic curation beyond automated QC thresholds.
- Non-video oceanographic data types (e.g., acoustic, chemical).
- Third-party archival systems beyond the KSO server infrastructure.

1.3 Stakeholders and their concerns

Stakeholder	Concerns & Usage of SAD Views
Marine Ecologists	• Verify data accuracy and species coverage (Information view) • Explore spatial/temporal visualizations (Functional & Deployment views)
Citizen Scientists	• Understand annotation workflow and UI components (Functional view) • Track progress and consensus rules (Information view)
ML Engineers & Developers	• Inspect component interactions and dataflows for preprocessing, training, and inference (Functional & Information views) • Manage code modules and dependencies (Development view)
System Administrators	• Plan deployment topology, storage allocation, and service-to-hardware mapping (Deployment view) • Automate backups and scaling workflows (Development view)
Sponsors & Funders	• Evaluate architectural extensibility and open-source licensing (Development view) • Assess risk-reduction measures and trade-off decisions (all views)

Table 1.1: Stakeholders and their primary concerns, mapped to SAD views.

2. References

[1] ISO/IEC/IEEE, "ISO/IEC/IEEE International Standard - Systems and Software Engineering – Life Cycle Processes – Requirements Engineering," 2018. ISO/IEC/IEEE 29148:2018(E), pages 1–104.

[2] N. Rozanski and E. Woods, *Software Systems Architecture: Working With Stakeholders Using Viewpoints and Perspectives*. Addison-Wesley Professional, 2011.

[3] V. Anton, J. Germishuys, P. Bergström, M. Lindegarth, and M. Obst, "An open-source, citizen science and machine learning approach to analyse subsea movies," *Biodiversity Data Journal*, vol. 9, p. e60548, 2021.

3. Glossary

AUV Autonomous Underwater Vehicle.

GPU Graphics Processing Unit.

KSO Koster Seafloor Observatory.

ML Machine Learning.

QC Quality Control.

REST API Representational State Transfer Application Programming Interface.

ROV Remotely Operated Vehicle.

SAD Software Architecture Description.

SRS Software Requirements Specification.

YOLOv3 "You Only Look Once" version 3.

4. Architectural Views

4.1 Functional View

4.1.1 Stakeholders' Uses of This View

The Functional View defines the system's runtime functional elements—their responsibilities, interfaces, and primary interactions—and shows how these collaborate to realize KSO's required capabilities.

This view addresses the concerns listed in Table 17-2 of Rozanski & Woods—functional capabilities, external interfaces, internal structure, and overall design philosophy—and serves as a cornerstone of the architectural description. Different stakeholder classes use it as follows:

- **Marine Ecologists (Users)**

  Verify that the system provides the analytic and export functions they need (e.g., species-count dashboards, spatial-distribution queries).

  Understand how data flows from ingestion through model inference to visualization, so they can plan their research workflows against the system's capabilities.

- **Citizen Scientists (Users)**

  Comprehend the clip-classification and annotation pipelines—how their inputs (clip labels, bounding boxes) propagate through the consensus aggregator to the database.

  See which external interfaces (web-UI endpoints, tutorial workflows) they will interact with during annotation tasks.

- **ML Engineers & Developers**

  Inspect component interactions (e.g., video-splitter → preprocessing service → training orchestrator → inference API) to validate separation of concerns, loose coupling, and latency-sensitive paths.

  Use sequence and activity diagrams derived from this view to plan unit and integration tests, and to guide implementation of connector protocols (RPC, messaging, REST).

- **System Administrators**

  Understand which functional elements (e.g., metadata service, model-serving endpoint, consensus engine) must be deployed on which hosts or containers.

  Plan monitoring and failure-recovery strategies by tracing runtime paths between elements and identifying critical connectors (message queues, API gateways).

- **Project Sponsors / Acquirers**

  Assess that all high-level functional requirements (ingestion, annotation, model inference, visualization) are covered by well-defined elements and interfaces.

  Evaluate architectural flexibility—how easily new functions (e.g., support for additional taxa) could be slotted into the existing component graph without wholesale redesign.

- **Testers**

  Derive test scenarios by walking through common use cases (e.g., "submit a clip → receive bounding-box review") against the interaction paths shown in component and sequence diagrams.

  Identify boundary points for black-box testing (REST endpoints, messaging topics) and white-box tests (component method contracts, error-handling flows).

By tailoring the level of abstraction and notation (UML component diagrams, sequence diagrams, or simplified "boxes-and-lines" sketches) to each group's background, the Functional View becomes an effective communication tool for both technical and non-technical stakeholders.

4.1.2 Component Diagram

We can see that Figure 4.1 shows us the high-level component structure of the Koster Seafloor Observatory (KSO), with all relationships indicated.

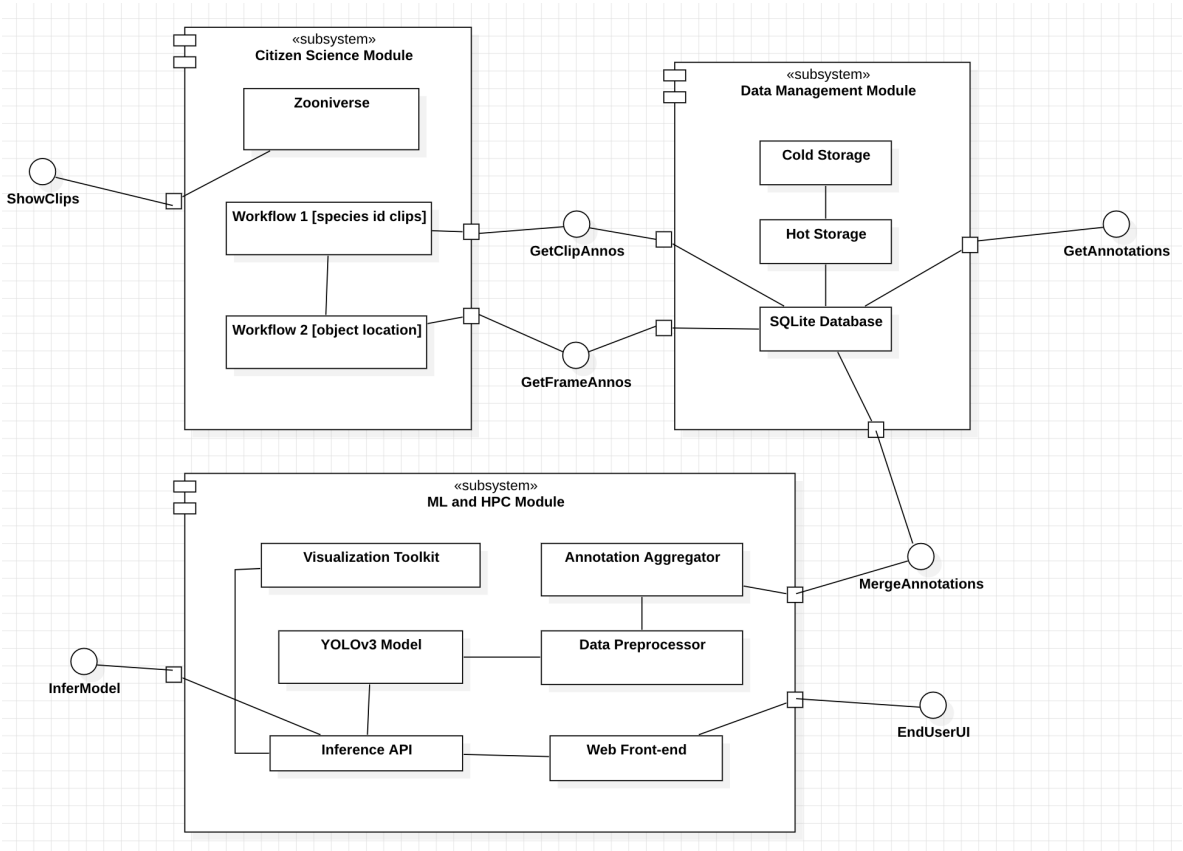

Figure 4.1: Component Diagram of KSO

Component Descriptions:

- **Data Management Module**
  - **Cold Storage** (`<<component>>`): long-term archive of raw subsea video files.
  - **Hot Storage** (`<<component>>`): HPC server holding the active working set of clips.

  - **SQLite Database** (`<<component>>`): stores metadata, clip/frame records, and annotation results.

- **Citizen Science Module**
  - **Zooniverse Project Site** (`<<component>>`): web portal exposing the two annotation workflows.
  - **Workflow 1: Species Identification** (`<<component>>`): presents 10 s clips for volunteer species labelling.
  - **Workflow 2: Object Localization** (`<<component>>`): allows volunteers to draw bounding boxes on frames.

- **Machine Learning & HPC Module**
  - **Annotation Aggregator** (`<<component>>`): merges citizen annotations based on agreement thresholds.
  - **Data Preprocessor** (`<<component>>`): performs frame-level tracking, augmentation, and normalization.
  - **YOLOv3 Model** (`<<component>>`): trains and tests detection on preprocessed frames.
  - **Inference API** (`<<component>>`, FastAPI): exposes prediction endpoints for the trained model.
  - **Web Front-end** (`<<component>>`, Streamlit): researcher-facing UI for uploading footage and tuning thresholds.
  - **Visualization Toolkit** (`<<component>>`): generates spatial and temporal distribution maps.

Interfaces (`<<provides>>`/`<<requires>>`)

- `Cold Storage` `<<provides>>` raw video clips to `Hot Storage`.
- `Hot Storage` `<<provides>>` clip metadata to `SQLite Database`.

- `SQLite Database` `<<provides>>` clips and metadata to `Workflow 1` and `Workflow 2`.
- `Workflow 1, Workflow 2` `<<provides>>` citizen annotations back to `SQLite Database`.
- `SQLite Database` `<<provides>>` aggregated frames to `Annotation Aggregator`.
- `Annotation Aggregator` `<<provides>>` cleaned, tracked frames to `Data Preprocessor`.
- `Data Preprocessor` `<<provides>>` training/test datasets to `YOLOv3 Model`.
- `YOLOv3 Model` `<<provides>>` inference service to `Inference API`.
- `Inference API` `<<provides>>` prediction endpoints consumed by `Web Front-end` and `Visualization Toolkit`.

4.1.3 Internal Interfaces

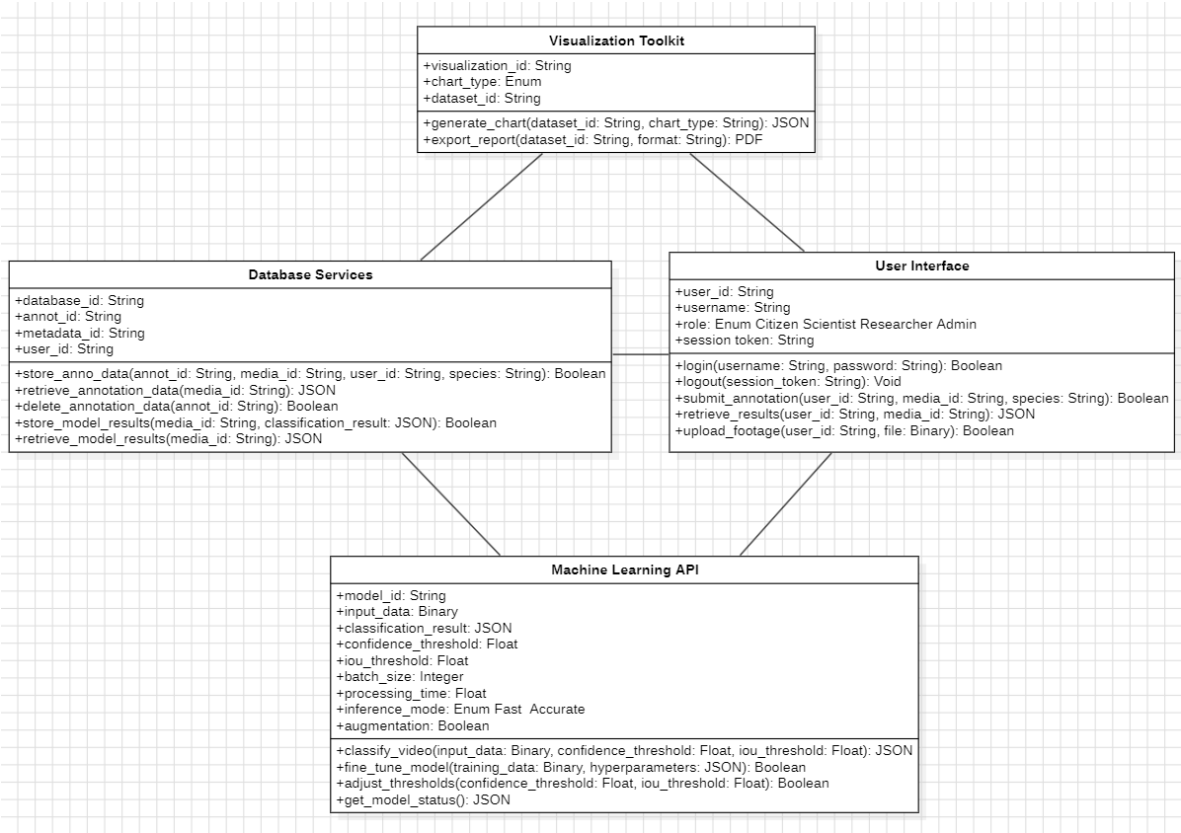

Figure 4.2: Internal Interfaces Class Diagram

User Interface Serving the User Interface's needs, this interface provides authentication (login/logout), annotation CRUD, result retrieval, and raw footage upload operations against the core Database Services. All methods accept JSON over REST and validate session tokens, enabling the front-end (web or Streamlit) to securely manage user sessions, persist annotations, fetch past results for display, and store newly uploaded videos without exposing database details to the client.

Database Service Interface This interface abstracts the orchestration between Database Services and underlying Storage Services by offering methods to store, retrieve, and delete media files; it also supports moving files between hot and cold storage tiers. Calls carry binary payloads alongside metadata in JSON and specify the

desired storage type, allowing KSO to optimize cost and performance by automatically archiving infrequently accessed footage while keeping active datasets on faster disks.

Machine Learning API Interface Acting as the bridge between Database Services and the Machine Learning API, this interface lets the database layer request video classification (with configurable confidence and IOU thresholds), store the JSON-formatted detection results, retrieve past model outputs, adjust inference parameters on the fly, and query model training status. All operations use REST+JSON, making it easy for the database to orchestrate inference workflows and record machine-generated annotations alongside citizen-science data.

Visualization Toolkit Interface This interface connects Database Services to the Visualization Toolkit by exposing operations to generate chart specifications (bar, line, heatmap, etc.) in JSON and export fully rendered reports in binary formats (PDF or HTML). By decoupling visualization concerns, KSO can produce interactive dashboards and print-ready summaries without embedding plotting logic in the core data services, ensuring a clean separation between data management and presentation layers.

4.1.4 Interaction Patterns

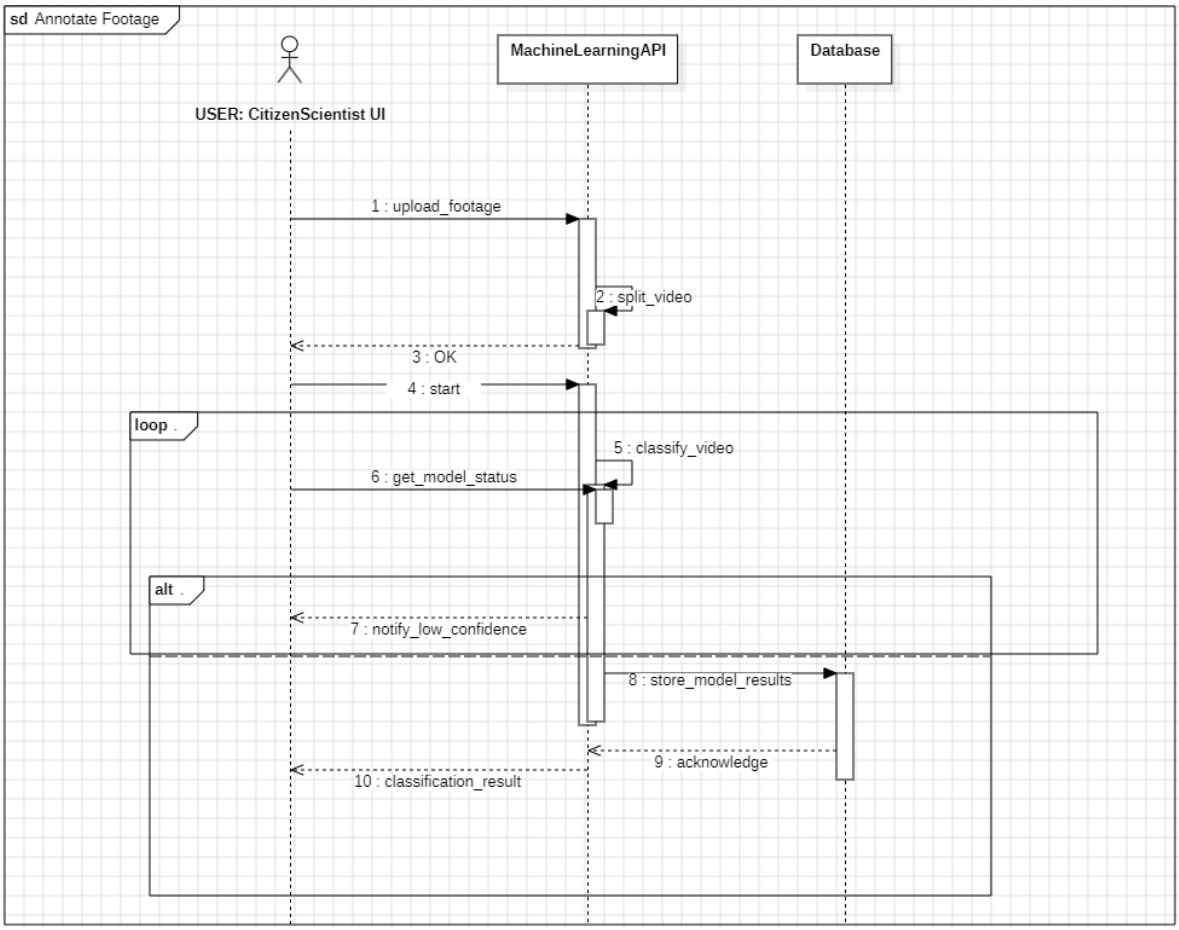

Figure 4.3: Sequence Diagram for Annotate Footage

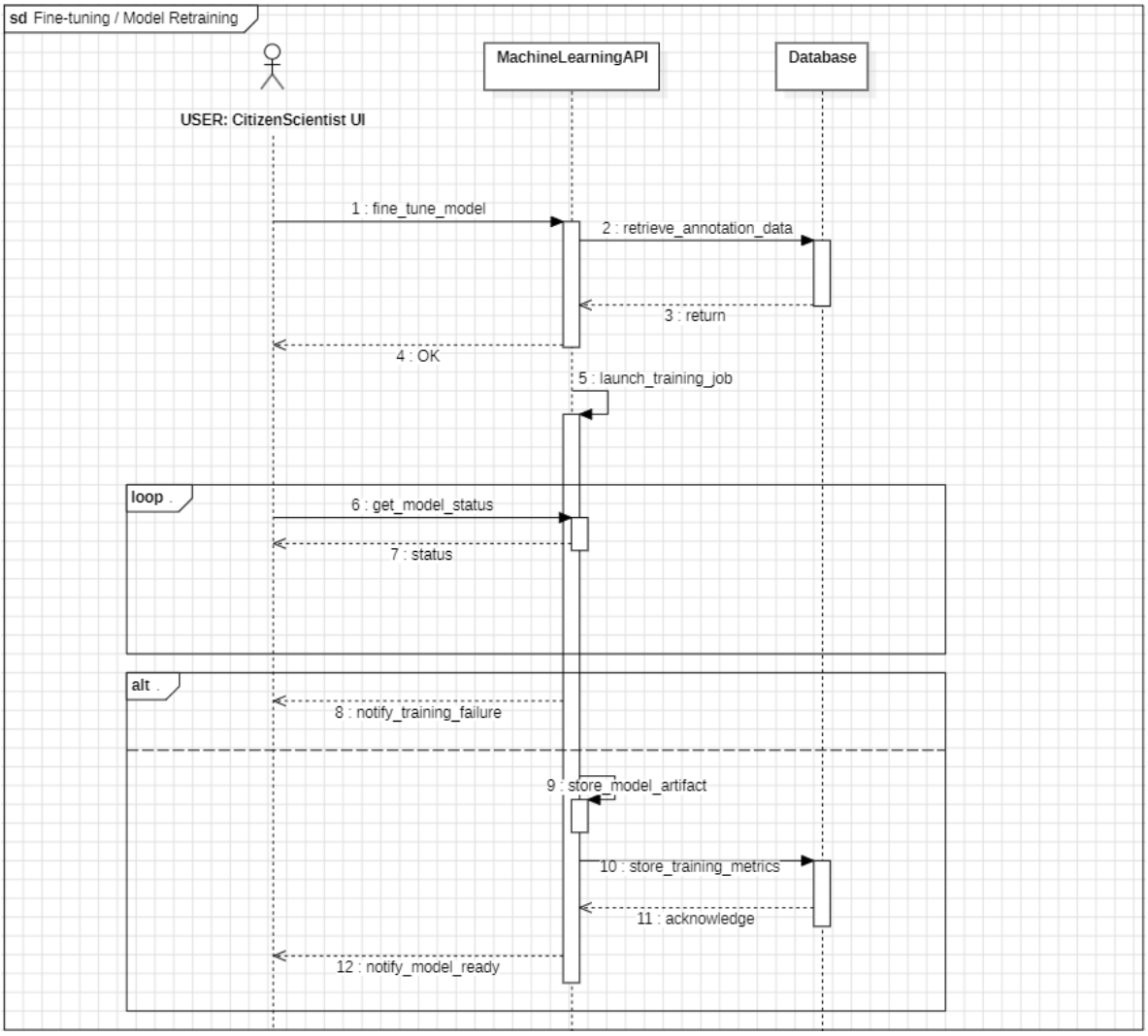

Figure 4.4: Sequence Diagram for Fine Tuning ML Models

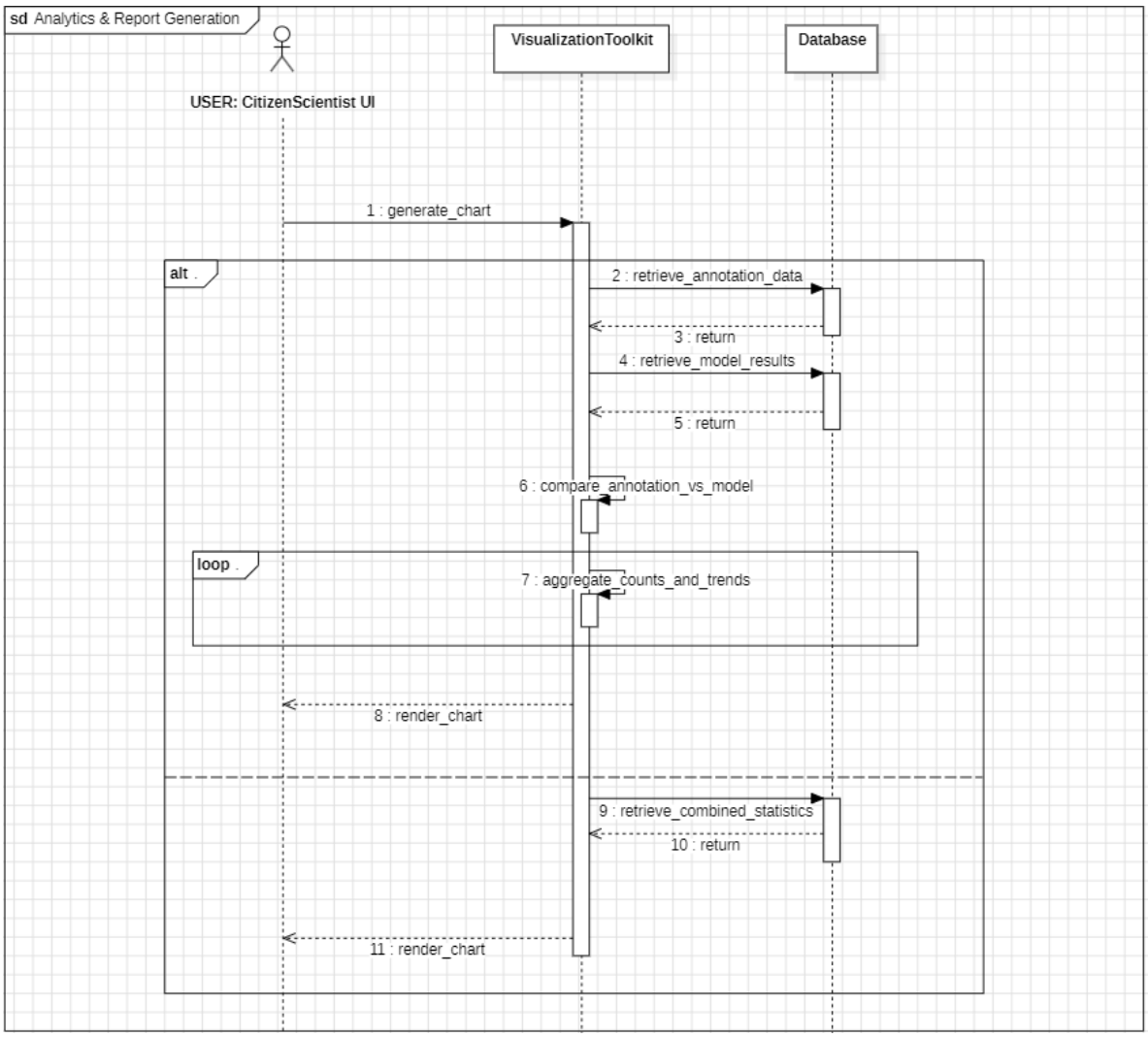

Figure 4.5: Sequence Diagram for Analytics and Report Generation

4.1.5 External Interfaces

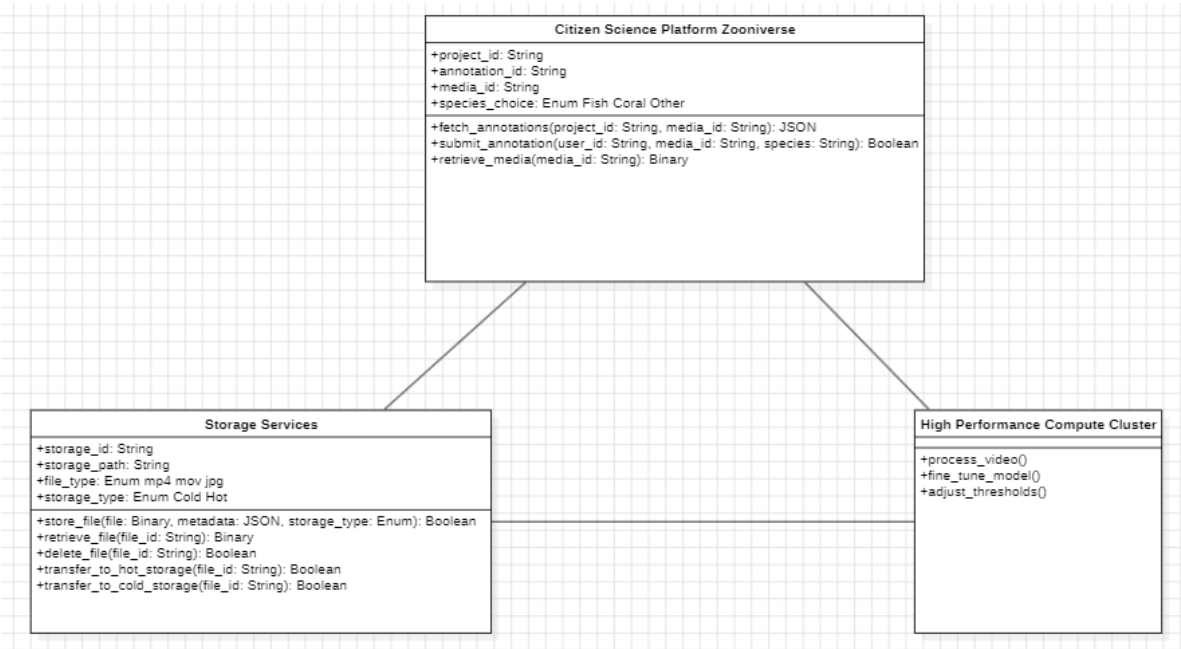

Figure 4.6: External Interfaces Class Diagram

Citizen Science Platform Interface This interface encapsulates all interactions with the Zooniverse citizen-science platform: it fetches project and workflow definitions (so that KSO knows which species and annotation tasks to present), uploads 10-second video clips for volunteer classification, submits individual user annotations back to Zooniverse, and retrieves both raw and aggregated annotation results. All operations use REST over HTTPS with JSON payloads and OAuth2 authentication, ensuring that KSO can seamlessly offload clip distribution and annotation collection to an established crowdsourcing service.

Storage Services This interface handles the push and query of metadata records to the Swedish National Data Archive using Darwin Core standards. It allows KSO to deposit or update metadata (e.g., location, date, camera settings) for each uploaded movie, perform spatial/temporal/keyword searches against archived datasets, and list all datasets associated with a given project. Communication occurs over HTTP(S)

using either JSON or XML (CSW/OAI-PMH style) with API-Key or client-certificate authentication, guaranteeing that KSO's scientific outputs remain interoperable with national biodiversity data infrastructures.

HPC Compute Cluster Interface This interface manages submission and monitoring of heavy compute jobs—such as ML training runs or batch inference—to an external high-performance computing cluster. KSO submits a JSON-encoded job configuration (container image, dataset locations, hyperparameters) and receives back a jobId; it can then poll that ID for status updates ("PENDING", "RUNNING", "DONE", "FAILED") and, upon completion, fetch the resulting model weights or inference outputs as binary artifacts. Transport may use a REST gateway or SSH/SFTP, ensuring that resource-intensive tasks scale without overloading the core KSO service.

4.1.6 Interaction scenarios

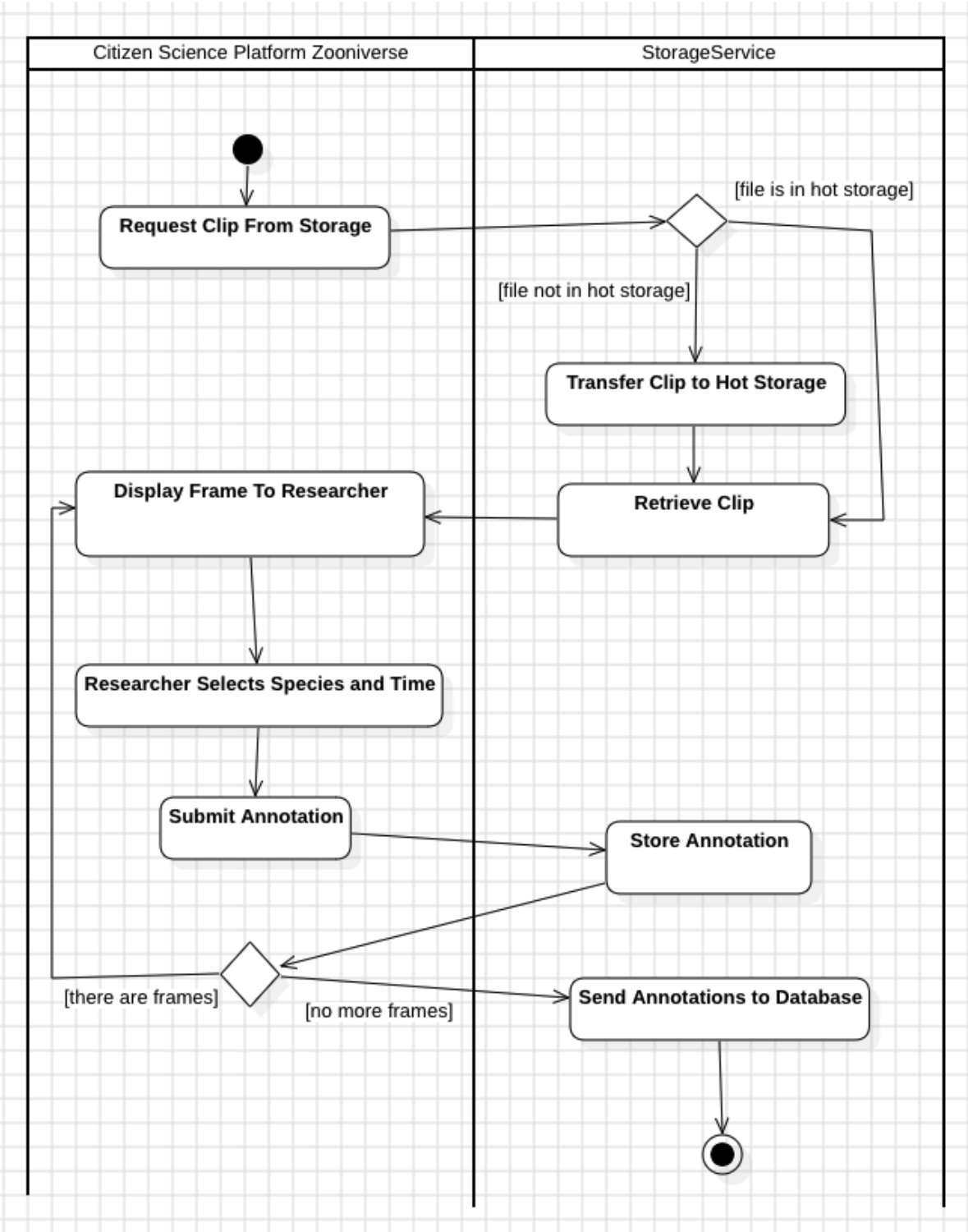

Figure 4.7: Activity Diagram for Annotate Footage

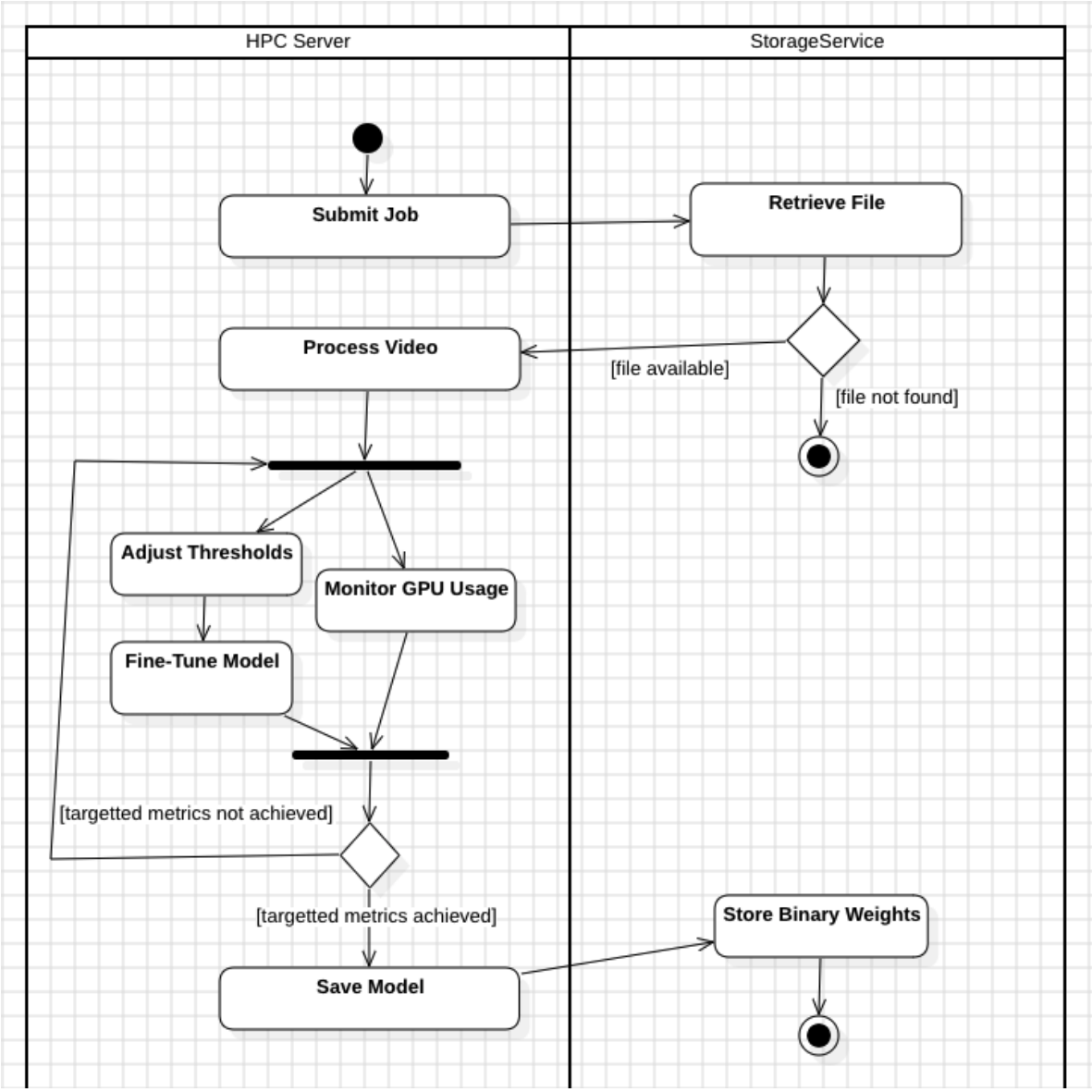

Figure 4.8: Activity Diagram for Fine Tuning ML Models

4.2 Information View

4.2.1 Stakeholders' Uses of This View

The Information View exposes the structure and operations on the key persistent "objects" in KSO. Its primary consumers are:

- **Database Administrators (DBAs)** – to understand the schema, relationships, and lifecycle of each table/class.
- **Backend Developers** – to see which methods they must implement in DAOs or repositories, and how entities relate.
- **System Integrators** – to discover which operations exist for creating, querying, and linking domain data (e.g. projects, movies, annotations).
- **QA/Test Engineers** – to identify boundary conditions (e.g. deleting a `Movie` must cascade or forbid if `Subject` exists).
- **Product Owners / Analysts** – to verify that each real-world concept (e.g. "Species count in a clip") is represented and manipulable in the API/DB.

4.2.2 Database Class Diagram

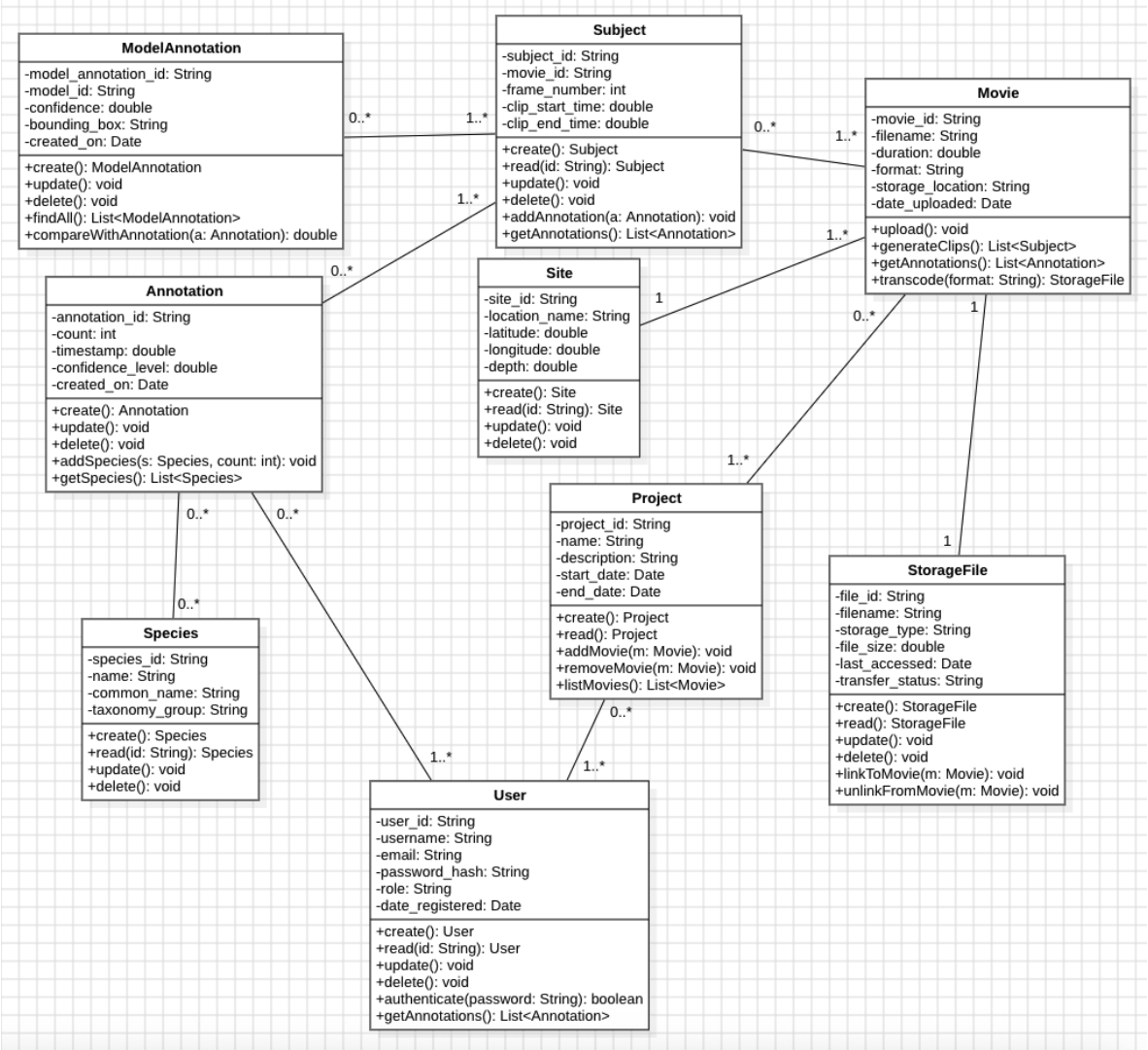

Figure 4.9: Database Class Diagram

Figure 4.9 models the eight core persistent entities of KSO and their navigations:

- **Project:** Encapsulates a research campaign; owns zero or more **Movie** instances and provides methods to add, remove and list them.
- **Site:** Represents a physical observation location; each Site hosts zero or more Movies.
- **Movie:** Stores video metadata (filename, duration, format, upload date, storage

URI). Offers CRUD plus domain methods to generate **Subject** clips, retrieve human **Annotation**s, and transcode into new **StorageFile**s.

- **Subject:** Defines a time-bounded segment of a Movie (frame number, start/end time). Connects to both human (**Annotation**) and algorithmic (**ModelAnnotation**) annotations.
- **Annotation:** Records a user-verified species count at a timestamp, links to one **User** and zero-to-many **Species** entries via its `addSpecies()` method.
- **ModelAnnotation:** Captures machine-generated detections (model name, confidence, bounding box). Can be compared to a human Annotation via `compareWithAnnotation()`.
- **StorageFile:** Represents raw or processed file artifacts for a Movie (size, transfer status, last accessed). Methods allow linking/unlinking to Movie records.
- **Species:** Centralizes taxonomy data; referenced by Annotation to record counts per species.
- **User:** Manages authentication/authorization and is the creator of zero or more Annotations.

4.2.3 Operations on Data

All classes support basic CRUD and finder methods, plus a few domain-specific navigations. Below we can list the key operations from the diagram:

Table 4.1: Operations on Data

Operation	Description
CreateAccount	Create: User Read: - Update: - Delete: -
DeleteAccount	Create: - Read: -

Continued on next page

Table 4.1 continued from previous page

Operation	Description
	Update: - Delete: User
AddNewAnnotation	Create: Annotation Read: Subject Update: Subject (addAnnotation) Delete: -
DeleteAnnotation	Create: - Read: - Update: - Delete: Annotation
AddNewSpecies	Create: Species Read: Annotation Update: Annotation (addSpecies) Delete: -
DeleteSpecies	Create: - Read: - Update: - Delete: Species
UploadMovie	Create: - Read: Movie Update: Movie (upload, generateClips, transcode) Delete: -
DeleteMovie	Create: - Read: - Update: - Delete: Movie

Continued on next page

Table 4.1 continued from previous page

Operation	Description
CreateSite	Create: Site Read: - Update: - Delete: -
DeleteSite	Create: - Read: - Update: - Delete: Site
CreateProject	Create: Project Read: - Update: Project (addMovie, removeMovie) Delete: -
DeleteProject	Create: - Read: - Update: - Delete: Project
LinkFileToMovie	Create: - Read: StorageFile, Movie Update: StorageFile (link/unlink to/from Movie) Delete: -
CreateStorageFile	Create: StorageFile Read: - Update: - Delete: -
DeleteStorageFile	Create: - Read: -

Continued on next page

Table 4.1 continued from previous page

Operation	Description
	Update: - Delete: StorageFile

4.3 Development View

4.3.1 Stakeholders' Uses of This View

The **Development View** of the Koster Seafloor Observatory (KSO) system provides a modular decomposition of software components, identifying architecturally significant subsystems and shared services. It is primarily used by stakeholders responsible for constructing, testing, integrating, and maintaining the system. This view supports clarity in development responsibilities, module independence, and traceable reuse of infrastructure components.

- **Software Developers** use this view to understand how the system is decomposed into subsystems (e.g., `AnnotationPlatform`, `MachineLearningEngine`, `DataManagement`) and which shared services (e.g., `MetadataService`, `TaskScheduler`) they can interact with. It guides the implementation of interfaces and ensures modular development.

- **Testers** rely on the Development View to design integration and module-level test strategies. By identifying dependencies between packages, they ensure correctness across subsystem boundaries and validate the behavior of shared services such as task scheduling or metadata queries.

- **System Integrators** consult this view to orchestrate component integration. It informs them of how subsystems relate and where orchestration logic is needed, especially around shared components like `StorageService` and `TaskScheduler`.

- **Project Managers and Team Leads** use this view to distribute development tasks and organize teams around package boundaries. It helps manage dependencies between subsystems, preventing unintentional coupling during parallel development.

- **Maintainers and Evolvers** benefit from the clear delineation of responsibilities among components. This view enables traceability of changes and supports safe evolution of individual subsystems without impacting unrelated parts of the architecture.

- **Security Engineers** may analyze this view to identify trust boundaries and evaluate access relationships. For example, ensuring that only authorized subsystems access the `StorageService` or submit jobs to the `TaskScheduler`.

4.3.2 Package Diagram

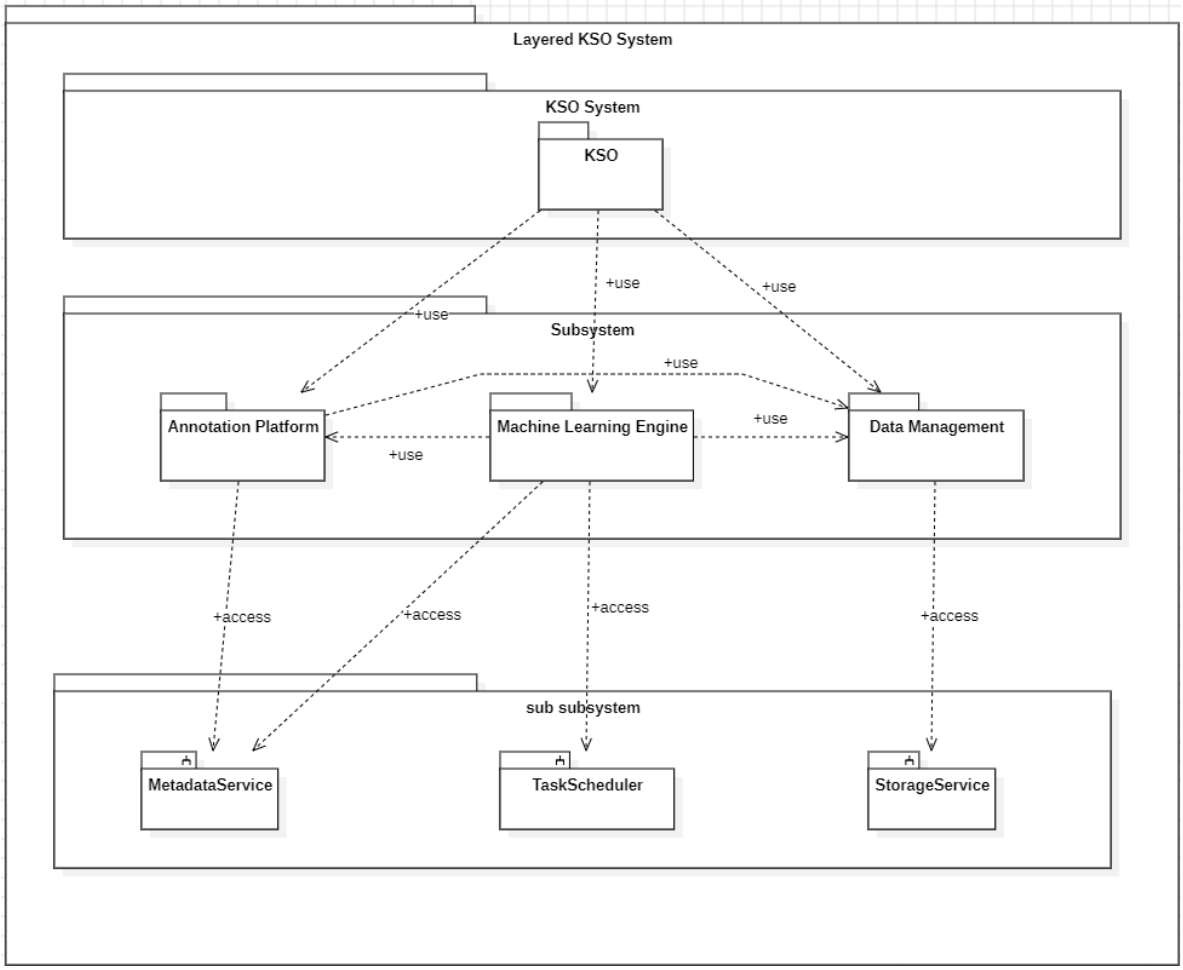

Figure 4.10: Package Diagram

At **Level 0**, the top-level package is the *Koster Seafloor Observatory (KSO)* system, which encapsulates all major software components responsible for the storage, annotation, and automated analysis of subsea video data. The system integrates citizen science contributions with machine learning pipelines to support large-scale marine biodiversity monitoring. This layered architecture promotes separation of concerns, scalability, and modular reuse.

At **Level 1**, the system is divided into three primary subsystems. The `AnnotationPlatform`

provides the web interface and functionality for citizen scientists to annotate video clips. It fetches clip and species metadata to display tasks and enables annotation submissions. The `MachineLearningEngine` subsystem manages the end-to-end machine learning workflow, including training models based on human annotations and applying those models to unlabeled video clips for automated species detection. It accesses shared services to retrieve input data and uses scheduling support for long-running background jobs. The third subsystem, `DataManagement`, orchestrates storage operations, including the movement of video files between cold (long-term) and hot (active) storage tiers. It ensures that relevant media are available to both the annotation and machine learning workflows.

At **Level 2**, three shared sub-subsystems provide core infrastructure services. The `MetadataService` enables querying and updating structured metadata related to media, species, annotations, and tasks. It is accessed by both the annotation and machine learning subsystems to coordinate consistent data usage. The `StorageService` is accessed solely by the `DataManagement` subsystem and provides an abstraction over physical storage systems, handling file retrieval and transfer. The `TaskScheduler` supports the `MachineLearningEngine` by managing background tasks such as model training and batch inference runs.

In terms of inter-package dependencies, the `MachineLearningEngine` retrieves human annotations from the `AnnotationPlatform` and accesses raw video clips via `DataManagement` for training and inference. Both these subsystems also interact with the `MetadataService` for task and species metadata. The `TaskScheduler` is used solely by the `MachineLearningEngine` for scheduling asynchronous jobs.

4.4 Deployment View

4.4.1 Stakeholders' Uses of This View

This deployment view addresses the concerns of the following stakeholders:

- **System Integrators** Rely on the deployment view to understand the distribution

of software components across hardware environments. They use this view to configure data flow pipelines (e.g., video transfer from field station to cold storage, annotation exchanges with Zooniverse) and ensure volume mounts and network links are correctly established for Docker-based components.

- **DevOps and Infrastructure Engineers** Use this view to plan infrastructure provisioning and resource allocation. They are responsible for deploying JupyterHub (Cloudina), mounting persistent storage (hot storage), and managing the execution environments on the HPC server. This view helps identify the containers (YOLO training, SQLite access) and their required volumes or access protocols (NFS, SCP, HTTPS).

- **Researchers and Data Scientists** Understand where and how various notebooks run, where to fetch or store processed data, and how to retrieve annotations. They use this view to know which scripts (e.g., `train_models.py`, `publish_observations.py`) run in which environments, and how the SQLite database integrates with their workflow.

- **Citizen Science Coordinators** While not directly managing deployments, they rely on this view to understand the flow of data to and from Zooniverse, ensuring the correct upload and retrieval mechanisms are in place for volunteer contributions.

- **Marine Biologists and Analysts** Benefit from understanding how raw footage collected in the field is integrated into the analysis pipeline. This view clarifies the relationship between field station outputs (e.g., .mp4, .csv files) and downstream tools used for annotation and ecological evaluation.

4.4.2 Deployment Diagram

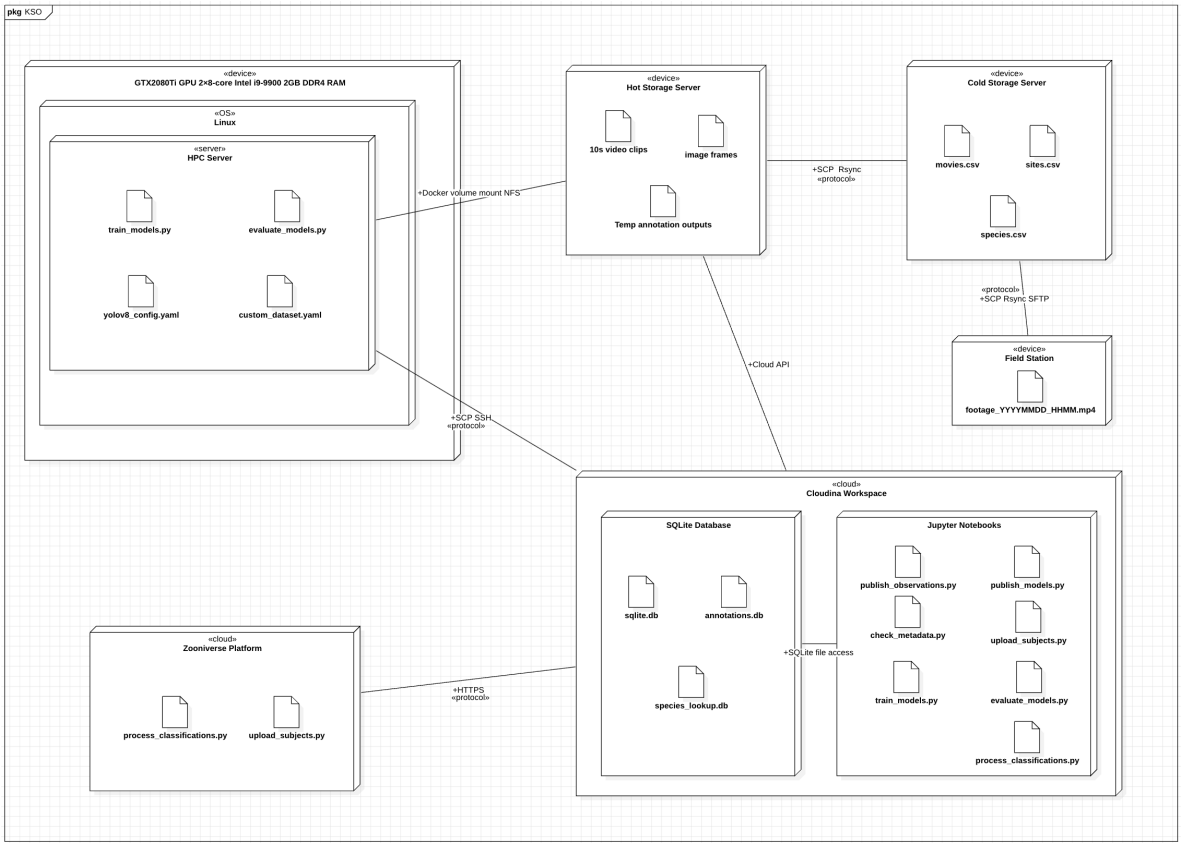

Figure 4.11: Deployment Diagram

The Deployment Diagram for the Koster Seafloor Observatory (KSO) illustrates the physical allocation of key software and data components across the system's infrastructure. It models the environment where system execution occurs, including both physical devices (e.g., Field Station, HPC Server) and virtual/cloud-based services (e.g., Zooniverse, Cloudina).

The field station collects raw footage and contextual metadata, which is transferred to a Cold Storage Server hosted at the University of Gothenburg for archival. A subset of this data is copied to the Hot Storage Server, which acts as short-term, fast-access storage for active analysis and annotation preparation. Jupyter Notebooks are executed

in the Cloudina Workspace (a JupyterHub-based environment), which accesses both the video clips and a local SQLite database used to manage metadata, annotations, and export-ready records.

The Zooniverse platform is used by citizen scientists to annotate subsea imagery. Cloudina scripts interact with Zooniverse via HTTPS using the `panoptes_client` API to upload frames and fetch classification results. These results are then used to prepare training datasets for machine learning models.

The HPC Server at Chalmers runs a containerized training environment (typically YOLOv5 or YOLOv8) and accesses training data via mounted volumes from the Hot Storage Server. The server does not contain its own persistent database and instead operates statelessly with inputs provided from Cloudina and outputs pushed back to shared storage.

Communication protocols such as rsync, scp, NFS, and RESTful HTTPS APIs are employed to enable data movement and service interaction across nodes. The SQLite database remains local to Cloudina, reinforcing the separation between storage (object-based) and structured metadata access (query-based).

This deployment view ensures modularity, reproducibility, and scalability in processing large-scale underwater ecological data across hybrid research infrastructures.

4.5 Design Rationale

- **Functional View**: We partitioned the system into three high-level modules—Data Management, Citizen Science, and Machine Learning—to maximize cohesion within each module while keeping coupling low. This modular structure lets us evolve or scale each capability independently as data volumes grow, at the cost of defining clear inter-module interfaces early on.

- **Information View**: We chose a Darwin Core–compliant SQLite relational schema [3] for storing movies, annotations, metadata, and model results. By adhering to a widely-adopted biodiversity data standard, we ensure seamless data sharing and reuse across research infrastructures, accepting moderate query-performance overhead in exchange for interoperability.

- **Development View**: We organized the codebase into a three-level modular hierarchy (Level 0: KSO, Level 1: subsystems like the Zooniverse connector, ML model, and API, Level 2: their internal packages) to enable parallel workstreams for data engineers, ML developers, and front-end teams. This increases upfront architectural complexity but dramatically reduces merge conflicts and integration risk as the project scales.

- **Deployment View**: We deploy each subsystem as a containerized microservice on our HPC cluster—Data Management on CPU nodes, ML inference on GPU nodes, and a lightweight Streamlit front-end—so that compute resources are matched to workload (high-throughput video processing vs. low-latency web UI). This delivers high performance and flexibility at the expense of more sophisticated orchestration and monitoring.

5. Architectural Views for Our Suggestions to Improve the Existing System

5.1 Functional View

5.1.1 Stakeholders' Uses of This View

- **Marine Ecologists**
  - Use the *Data Catalog Service* to search, filter and retrieve clips by location, date, or species metadata.
  - Leverage the *Visualization Toolkit* functions to generate spatial–temporal maps of species occurrences.
- **Citizen Scientists**
  - Rely on the *Tutorial Manager* to access step-by-step guidance and contextual help before annotating.
  - Submit annotations through *Workflow 1* and *Workflow 2* and receive quality feedback via the *Quality Control Service*.

- **ML Engineers & Developers**
  - Invoke the *Hyperparameter Tuner* to automate model parameter sweeps and improvements.
  - Register and retrieve model artifacts through the *Model Registry* for versioning, rollback, and A/B testing.
  - Call the *Inference API* for batch or online predictions in downstream applications.
- **System Administrators**
  - Schedule and monitor the *Backup Manager* to ensure hot/cold storage snapshots and disaster-recovery workflows.
  - Configure and enforce access control via the *Auth Service*.
  - Dispatch alerts and system notifications using the *Notification Service*.
- **Sponsors & Funders**
  - Evaluate the *Quality Control Service* and *Backup Manager* as risk-mitigation mechanisms.
  - Assess the *Data Catalog* and *Model Registry* for system extensibility, reproducibility, and governance.

5.1.2 Component Diagram

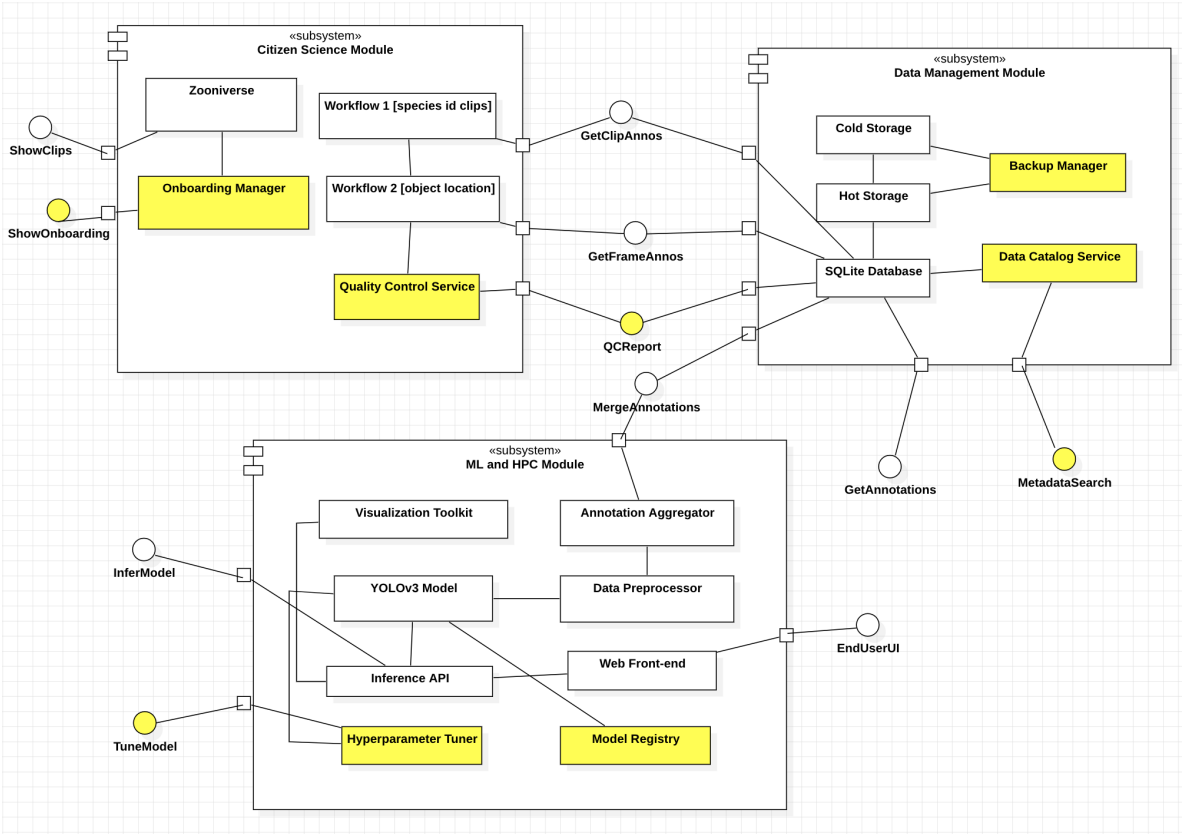

Figure 5.1: Improved Component Diagram

We made the following additions to our component diagram, as can be seen from Figure 5.1:

Data Management Module

- **Data Catalog Service** (`<<component>>`)
  - Provides searchable index of clip/frame metadata
  - Interface: ≪`MetadataSearch`≫
- **Backup Manager** (`<<component>>`)

  - Automates snapshots and replication of hot/cold storage
  - Interface (internal): ≪`Backup`≫

Citizen Science Module

- **Quality Control Service** (`<<component>>`)
  - Applies consensus QC rules and flags low-confidence annotations
  - Interface: ≪`QCReport`≫
- **Tutorial Manager** (`<<component>>`)
  - Hosts onboarding tutorials and context-sensitive help
  - Interface: ≪`ShowOnboarding`≫

Machine Learning & HPC Module

- **Hyperparameter Tuner** (`<<component>>`)
  - Automates parameter sweeps and optimization
  - Interface: ≪`TuneModel`≫
- **Model Registry** (`<<component>>`)
  - Version-controls trained models and artifacts
  - Interface (internal): ≪`ModelVersion`≫

These additions boost discoverability, reliability, quality control, and manageability across all modules.

5.1.3 Internal Interfaces

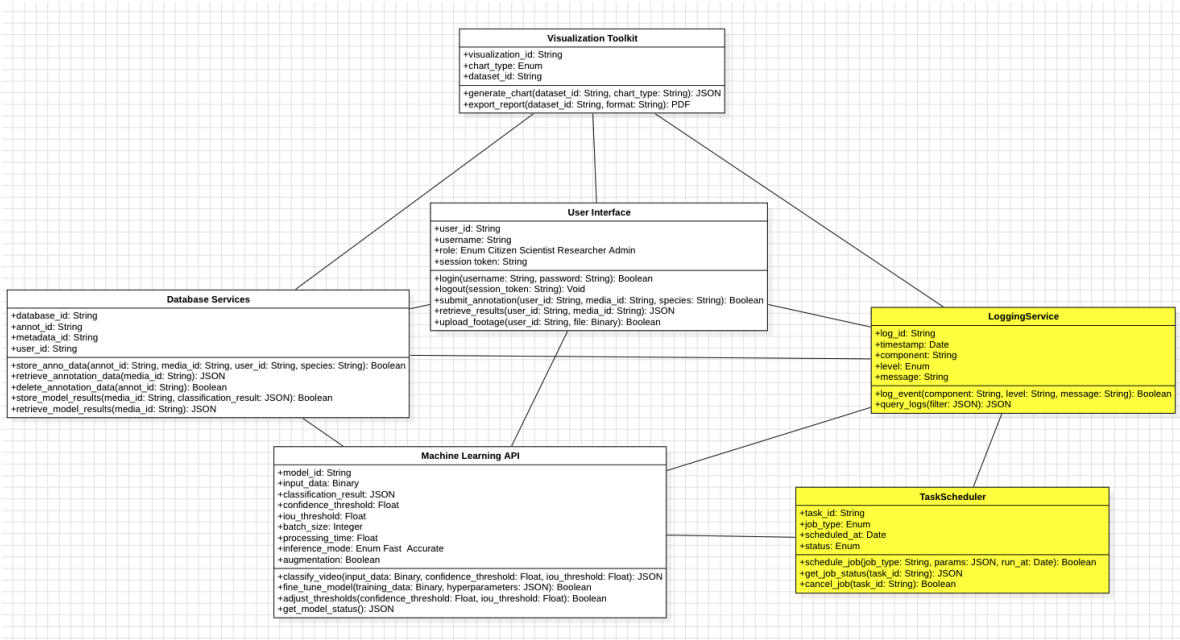

Figure 5.2: Improved Internal Interfaces Class Diagram

As can be seen from the Figure 5.2, we added the following internal interfaces:

TaskScheduler Interface: This interface handles the queuing, scheduling and lifecycle management of all asynchronous, compute-intensive jobs (e.g. video classification, model fine-tuning, report export). KSO submits a JSON payload containing the `jobType`, parameters and desired start time and receives back a `taskId`; it can then poll that `taskId` via REST (or consume status events via a message queue) to observe transitions (“`PENDING`”, “`RUNNING`”, “`COMPLETED`”, “`FAILED`”), cancel pending work, or retrieve error details. Under the hood it persists tasks in a durable queue (e.g. RabbitMQ or a database table) to guarantee reliable delivery, survive restarts, and decouple heavy workloads from the core service.

LoggingService Interface: This interface collects, stores and serves all operational and audit logs emitted by KSO components (UI, Database Services, Machine Learning API, TaskScheduler, Visualization Toolkit). Each subsystem invokes a REST or gRPC call with a JSON record—containing `timestamp`, `component`, `level` and `message`—and receives a Boolean acknowledgment. Consumers (e.g. dashboards or alerting tools) can query the service with JSON-encoded filters (time ranges, severity, component) to retrieve matching entries. The LoggingService backs onto an append-only store with configurable retention and guarantees write latency $\leq$ 2 ms and query latency $\leq$ 200 ms for recent data.

5.1.4 Interaction Patterns

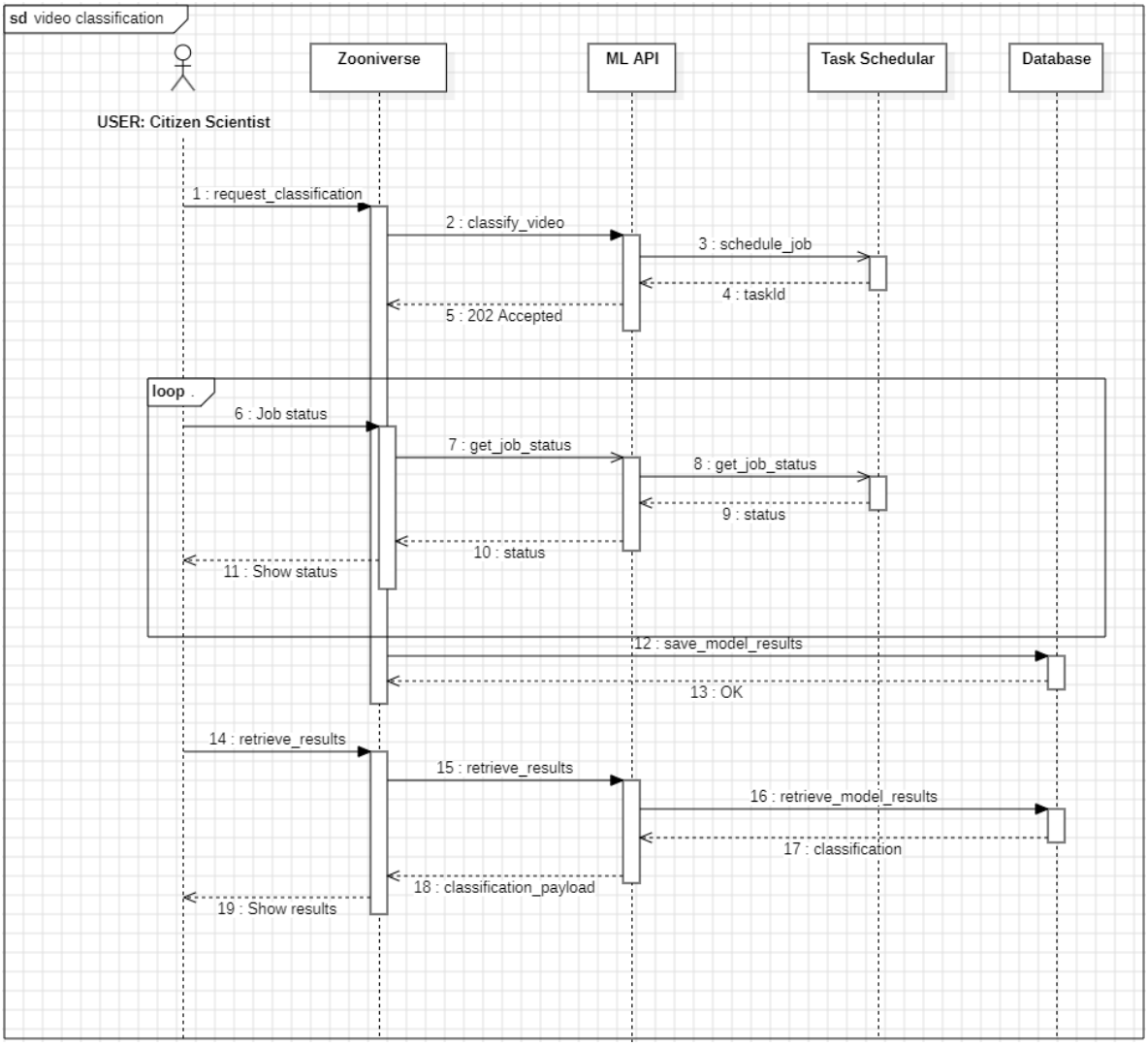

Figure 5.3: Suggested Sequence Diagram for Video Classification

5.1.5 External Interfaces

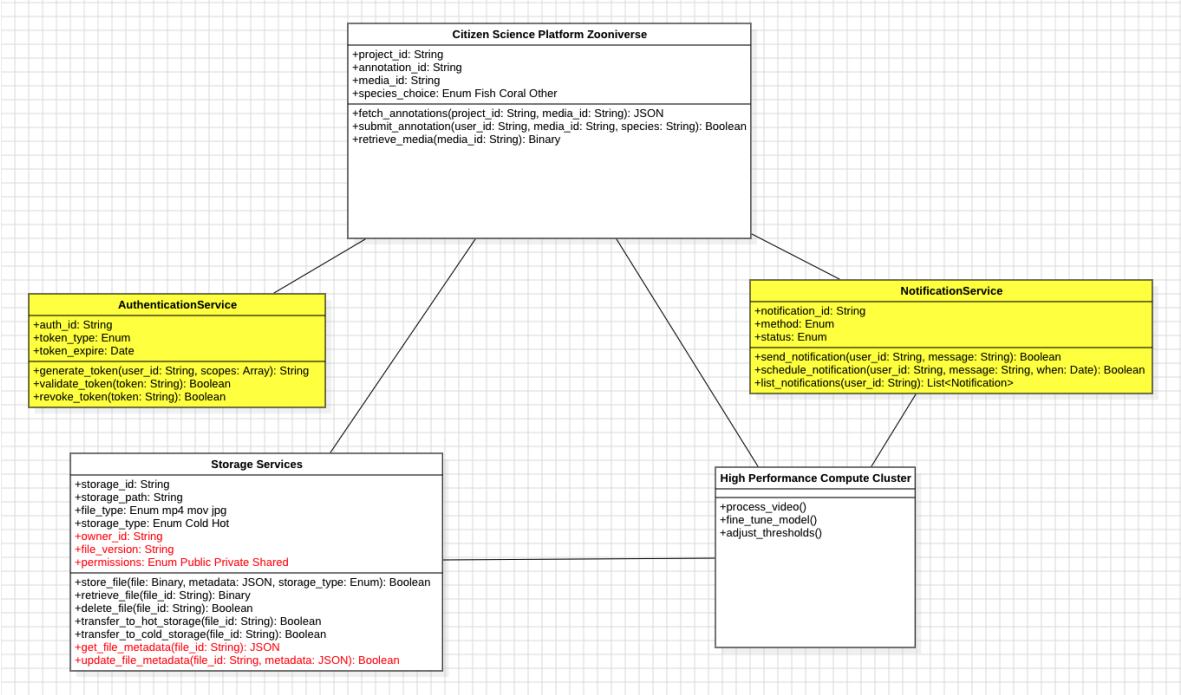

Figure 5.4: Improved External Interfaces Class Diagram

Figure 5.4 shows our suggestions, let's clarify each of them:

AuthenticationService Interface: This external interface provides secure token issuance and validation for all KSO clients (web UI, API gateways, HPC submissions, etc.). KSO issues a REST POST to the Authentication Service with a JSON payload containing `userId` and requested `scopes`, and receives back a JSON response with `accessToken`, `tokenType` (e.g. "Bearer"), and `expiresIn`. Subsequent calls to any protected KSO endpoint include this token in the `Authorization` header. The interface also exposes a validation endpoint where KSO can POST a token and receive a Boolean success flag, and a revocation endpoint which invalidates a token before its expiry. Transport is HTTPS, and the service typically implements OAuth 2.0 or SAML under the hood to ensure tokens cannot be forged or replayed.

NotificationService Interface: This external interface enables KSO to push alerts and updates to end users or downstream systems via Email, SMS or Webhook. KSO sends a REST POST with a JSON body containing `recipientId`, `method` (Email/SMS/Webhook), `message`, and an optional `scheduleAt` timestamp; the service returns a JSON containing `notificationId` and initial `status` ("PENDING"). KSO can then GET `/notifications/{notificationId}` to retrieve status transitions ("SENT", "FAILED"), or DELETE the endpoint to cancel pending deliveries. The service also supports a bulk-send endpoint for batch notifications and can call back to a KSO-provided webhook URL upon final delivery, ensuring reliable, asynchronous alerts for long-running tasks.‘‘

StorageService Improvements: StorageServices were also improved to enable metadata access, file versioning, permissions, and user-based file listings. The upgrade enables support for handling higher volumes of data more effectively with fine-grained control over file ownership, visibility, and lifecycle. These features are aligned with cloud storage best practices and are required to offer secure, multi-user environments.

5.1.6 Interaction scenarios

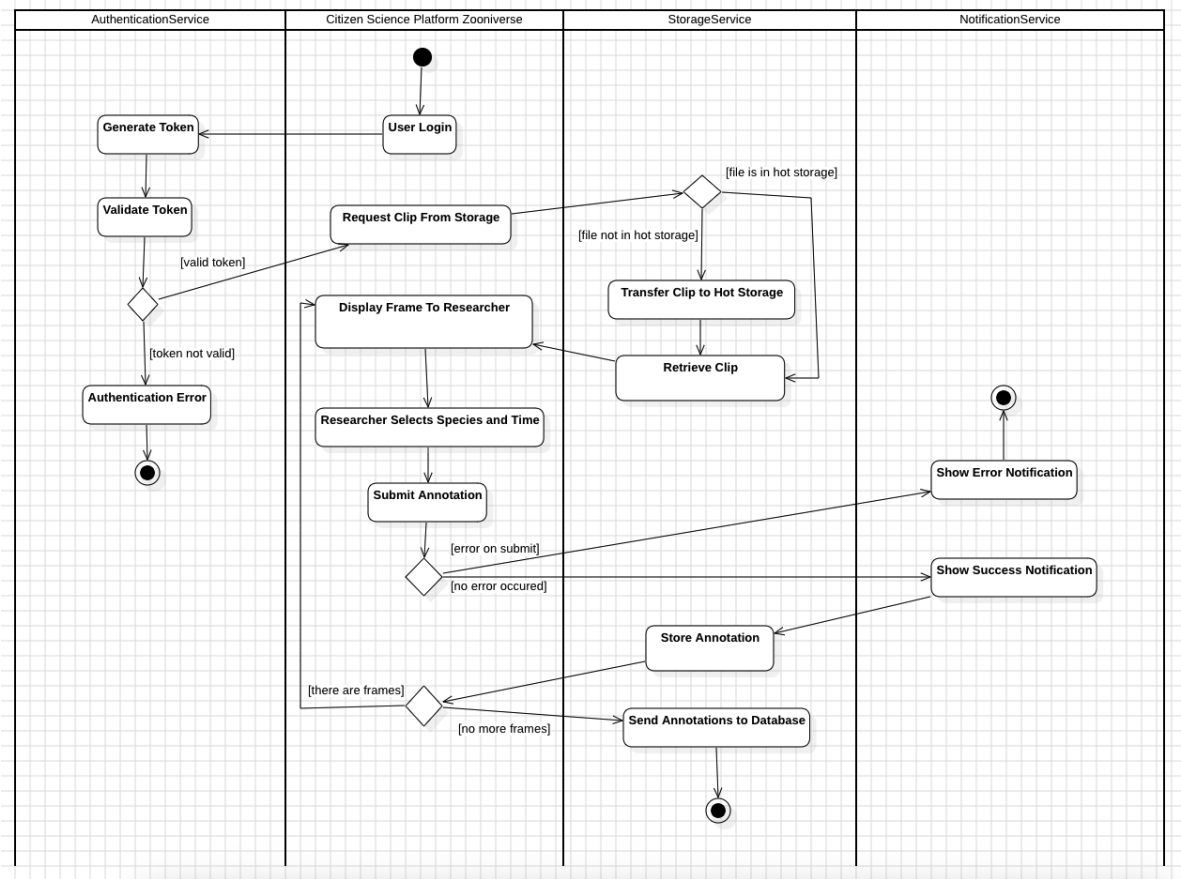

Figure 5.5: Suggested Activity Diagram for Annotate Footage

5.2 Information View

5.2.1 Stakeholders' Uses of This View

The Information View models the structure, attributes, and relationships among the persistent entities of the Koster Seafloor Observatory (KSO) system. With our suggestions, it will capture not only core domain objects but also nuanced metadata and analytical indicators (e.g., bounding boxes, annotation counts, extinction tags). The following stakeholders use this view as follows:

- **Database Administrators (DBAs)** – rely on this view to understand both high-level relationships and low-level column details such as `annotation_count`, `bounding_box`, or `transfer_details`. The inclusion of fields like `metadata_note` and `affiliation` helps guide normalization decisions and performance indexing for frequent queries or filtering.

- **Backend Developers** – use this diagram to implement data-access layers (e.g., DAOs, repositories). With the new attributes, developers can enrich APIs that support filtering by user `affiliation`, flag annotations by source (e.g., manual vs. automated), or fetch extinct species data for dynamic UI filtering or validation.

- **System Integrators** – benefit from knowing how systems interlink at both object and attribute levels. For example, the ability to link `ModelAnnotation` to bounding box coordinates and associate it with a specific `Subject` allows integration with external ML pipelines, which need precise frame-level mapping and model-generated coordinates.

- **QA/Test Engineers** – need to verify edge cases such as deletion of a `Species` that's marked as `Extinct`, or handling of unexpected `annotation_source` values. The `annotation_count` field adds another test vector, ensuring consistency between the annotation list and the summary count stored in the `Subject`.

- **Product Owners / Analysts** – gain confidence that new user stories and analytical goals are covered in the schema. The ability to attach `metadata_note` to `Movie` entries, store `affiliation` per `User`, or differentiate between annotation origins (human vs. model) supports reporting, traceability, and accountability across the scientific pipeline.

- **ML Researchers / Data Scientists** – although not previously listed, this enhanced model now supports their needs. Bounding boxes (`bounding_box0`...`bounding_box3`), `confidence`, and `compareWithAnnotation()` enable direct evaluation of model performance, and the separation of `annotation_source` provides a clean way to compare human vs. model accuracy.

5.2.2 Database Class Diagram

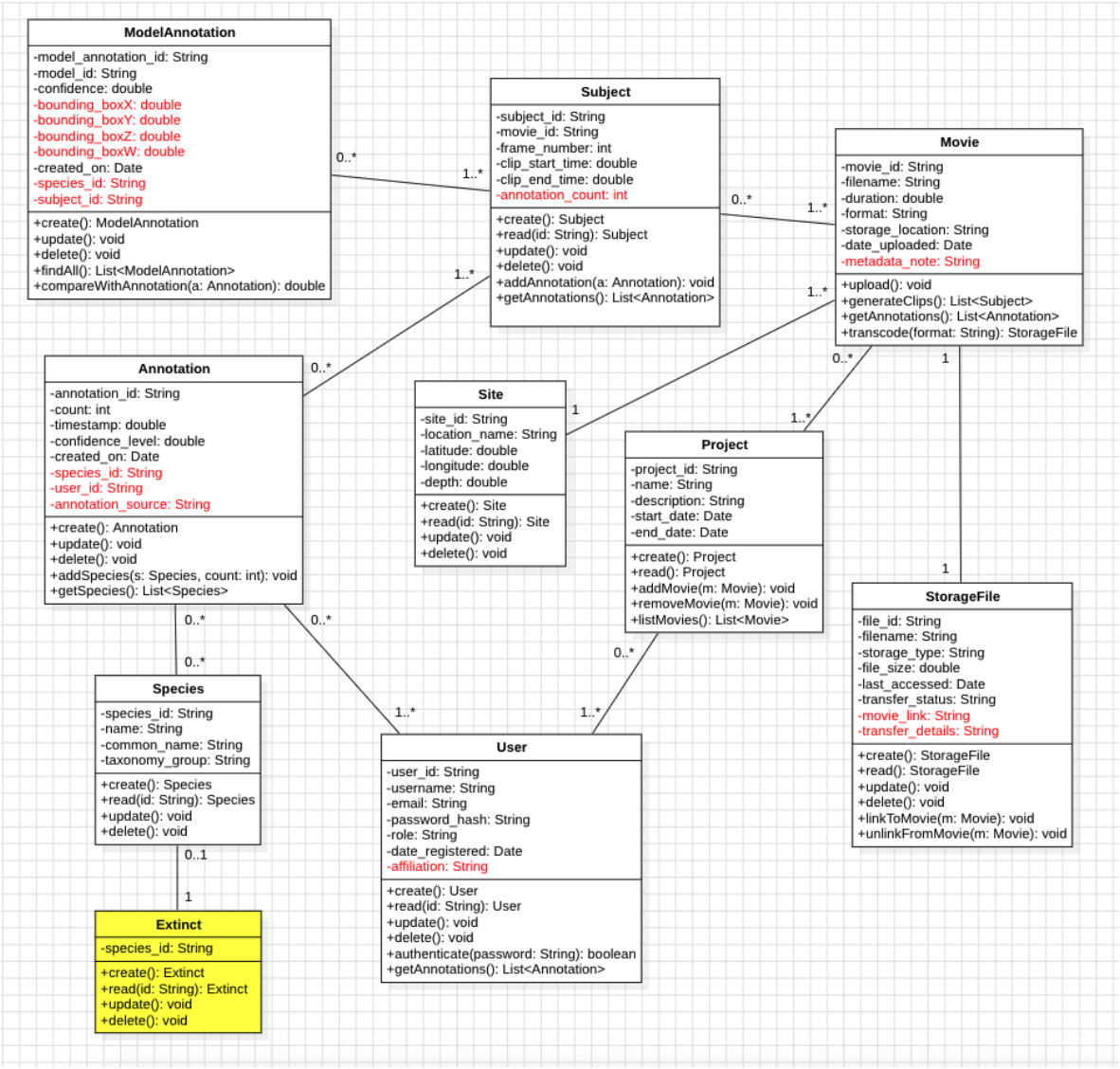

Figure 5.6: Improved Database Class Diagram

The database class diagram has been extended to support richer domain semantics, improved query performance, and greater analytical depth. These improvements reflect both functional enhancements and new modeling decisions aligned with KSO's evolving requirements.

- **Support for Bounding Boxes and Model Confidence:** The `ModelAnnotation` class was extended with bounding box coordinates and a confidence score to enable integration with machine learning pipelines. These additions allow for precise spatial localization of detected features and support downstream evaluation processes, such as comparing model results with human annotations.

- **Direct Linking Between Model Outputs and Video Segments:** A direct reference from `ModelAnnotation` to `Subject` was added to allow model predictions to be explicitly associated with individual video clips. This simplifies traceability and improves performance when querying model results for a specific temporal segment.

- **Annotation Count Tracking:** A new attribute in the `Subject` entity records the total number of linked annotations. This denormalized value supports faster rendering of summary views or dashboards and reduces the need for expensive aggregate queries.

- **Metadata Annotations for Movies:** The `Movie` class now includes a free-text metadata field that allows users to annotate video files with contextual notes, warnings, or observational remarks. This enriches the interpretability of raw media assets.

- **Enhanced File-Movie Association Details:** The `StorageFile` class was extended to include both a logical reference to the associated movie and detailed transfer metadata. This supports more transparent storage management and can assist in tracking file provenance or delivery status across systems.

- **Annotation Provenance and User Context:** The `Annotation` class now captures both the user who submitted the annotation and the origin of the data (e.g., manual vs. automated). This allows system users to differentiate between human-labeled and auto-generated annotations and apply filters accordingly.

- **User Affiliation Management:** An `affiliation` attribute was added to the

`User` entity to record institutional or organizational context. This is useful for grouping contributors by research group or university, and for generating team-level summaries.

- **Extinction Status for Species:** A new entity was introduced to record species that are extinct. It is linked to `Species` in a one-to-one relationship and enables special-case handling, such as flagging annotations that reference extinct species or supporting conservation-focused queries.

These improvements collectively enhance the schema's expressiveness, allow for better system performance, and facilitate integration with both scientific workflows and modern analytical tools.

5.2.3 Operations on Data

Table 5.1: Extended Operations on Data

Operation	Description
RecordModelBoundingBox	Create: ModelAnnotation (bounding_box_*) Read: ModelAnnotation Update: ModelAnnotation (bounding_box_*) Delete: -
LinkModelToSubject	Create: ModelAnnotation (subject_id) Read: Subject Update: - Delete: -
TrackAnnotationCount	Create: - Read: Subject (annotation_count) Update: Subject (increment on addAnnotation) Delete: -

Continued on next page

Table 5.1 continued from previous page

Operation	Description
AddMovieMetadataNote	Create: - Read: Movie Update: Movie (metadata_note) Delete: -
LinkMovieInStorage	Create: - Read: StorageFile (movie_link) Update: StorageFile (movie_link, transfer_details) Delete: -
TrackAnnotationSource	Create: Annotation (annotation_source) Read: Annotation Update: Annotation (source tag change) Delete: -
AttachUserAffiliation	Create: User (affiliation) Read: User Update: User (affiliation) Delete: -
MarkSpeciesAsExtinct	Create: Extinct (species_id) Read: Species, Extinct Update: Extinct Delete: Extinct

5.3 Development View

5.3.1 Stakeholders' Uses of This View

- **Software Developers** benefit from the addition of the `LoggingService`, which allows them to emit structured logs during feature development, easing debug-

ging and operational diagnostics. It complements the architectural modularity by offering a standardized logging interface across subsystems.

- **Testers** gain improved visibility into subsystem interactions, as runtime logs can now be analyzed to verify integration correctness and diagnose errors across shared services like the `MetadataService` and `TaskScheduler`.
- **System Integrators** use the `LoggingService` to monitor integration events and validate the operational flow between modules. This ensures smooth orchestration of subsystems such as `MachineLearningEngine` and `DataManagement`.
- **Project Managers and Team Leads** are able to use logs generated across subsystems for progress tracking, error trend analysis, and operational health assessments, especially during staging and deployment phases.
- **Maintainers and Evolvers** gain a valuable source of historical and runtime information through logs, which assist in debugging regressions, understanding component behavior, and making informed decisions when evolving the system.
- **Security Engineers** now have access to centralized logs that can be used for forensic analysis, access monitoring, and compliance verification. The presence of a unified logging mechanism simplifies the enforcement of audit policies across the architecture.

5.3.2 Package Diagram

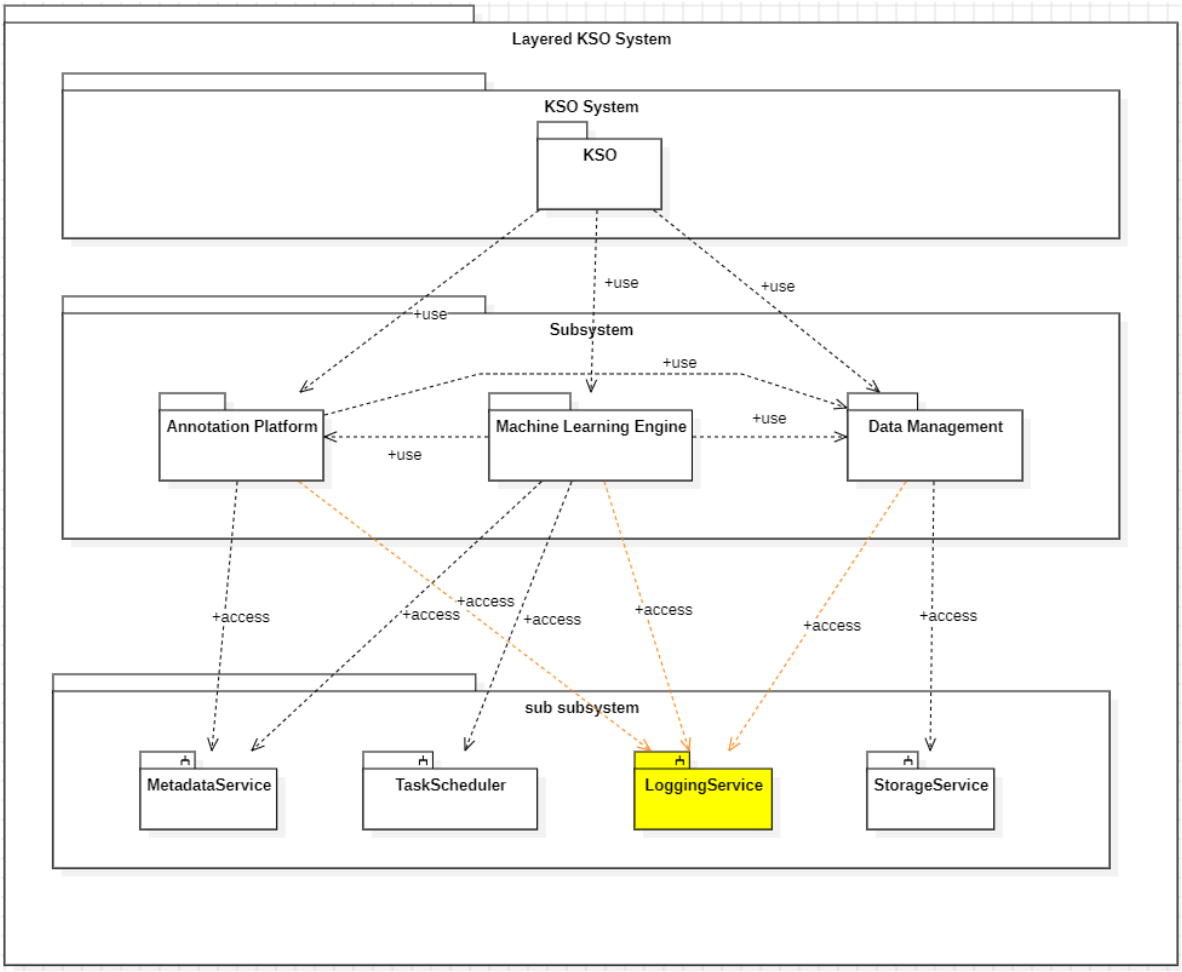

Figure 5.7: Improved Package Diagram

To improve traceability, observability, and operational transparency across the KSO system, an additional cross-cutting sub-subsystem named `LoggingService` is introduced at Level 2. This service provides unified logging capabilities to all subsystems, supporting error tracking, audit trails, performance diagnostics, and system monitoring.

At **Level 2**, the `LoggingService` becomes a shared technical service accessed by all three Level 1 subsystems. The `AnnotationPlatform` uses it to record user activities and platform-level exceptions. The `MachineLearningEngine` logs training progress,

inference metrics, and pipeline errors, while `DataManagement` logs file transfer events, storage health checks, and orchestration outcomes.

This improvement results in the following updated dependencies:

- `AnnotationPlatform` → `LoggingService`
- `MachineLearningEngine` → `LoggingService`
- `DataManagement` → `LoggingService`

This change reinforces modularity by isolating operational concerns into a dedicated service and ensures that logs across the system follow a consistent schema, aiding future maintainability and debuggability.

5.4 Deployment View

5.4.1 Stakeholders' Uses of This View

- **System Integrators** Use this view to configure connections between the Unified Storage Area, Cloudina Workspace, and HPC Server. They ensure proper mounting and API routing without direct database coupling.
- **DevOps and Infrastructure Engineers** Plan and manage the deployment of containerized services (e.g., JupyterHub, Metadata API Gateway), storage mounts, and communication protocols such as HTTP, NFS, and SCP.
- **Researchers and Data Scientists** Leverage the Metadata API to access annotation and species metadata from notebooks, without managing raw SQL or file-based database access. They use shared storage for consistent access to video and training datasets.
- **Citizen Science Coordinators** Understand the Zooniverse integration path and how annotated data is programmatically fetched, validated, and incorporated into training workflows via the metadata pipeline.

- **Marine Biologists and Analysts** Gain insight into how field-collected footage (.mp4, .csv) moves from storage to annotation, training, and evaluation, supported by a unified and traceable architecture.

5.4.2 Deployment Diagram

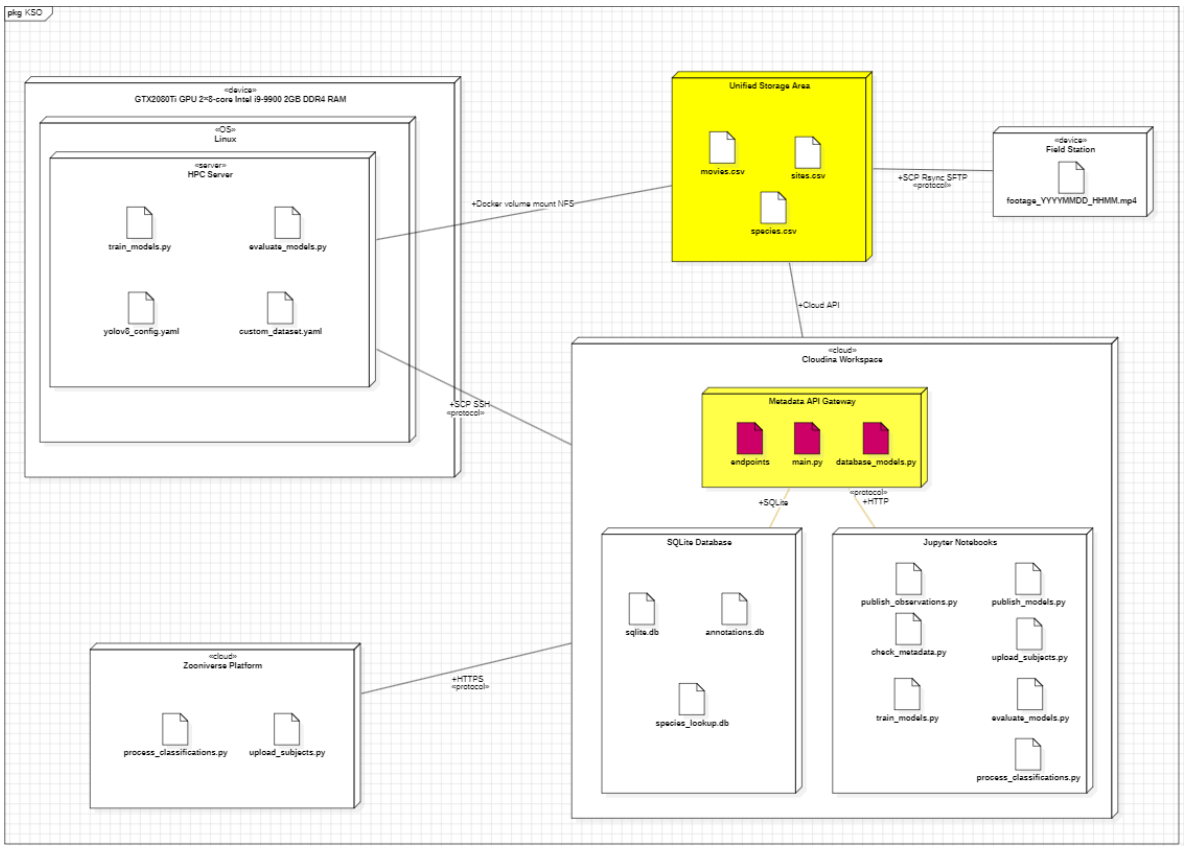

Figure 5.8: Improved Deployment Diagram

In the original deployment model, data was separated between Cold Storage (for long-term archival) and Hot Storage (for short-term active use). This introduced unnecessary data duplication, transfer overhead, and potential synchronization issues.

As an improvement, these storage tiers have been merged into a single Unified Storage Area, which retains the semantic distinction between archival and active-use files via directory structure or object tagging (e.g., cold/, hot/, processed/). This

storage area is mounted as a shared volume or networked filesystem accessible from both the Cloudina Workspace and the HPC Server, ensuring that; Annotated media, video clips, and derived datasets remain in a consistent, centralized location. Notebooks, training containers, and export processes can access files without manual copying.

Storage management policies (e.g., archival, deletion, versioning) are unified and simplified. This consolidation streamlines operations and enables efficient, reproducible workflows across the system.

In the improved architecture, the Cloudina Workspace no longer accesses the SQLite database directly. Instead, all structured metadata interactions (e.g., querying species, adding annotations, validating sites) are handled through a new internal component: the Metadata API Gateway.

This gateway is deployed within the same environment as the Jupyter notebooks, exposing RESTful endpoints to the notebooks and other internal clients. The API manages all read/write operations to the SQLite database, handling validation, schema abstraction, and request sanitization.

This change brings several benefits; Decouples metadata access from file-based operations, improving security and maintainability. Supports future extensions, such as remote dashboards or data export services, without exposing the raw database. Enforces API contracts, reducing the risk of query mismatches or schema drift.

Together, these improvements form a more modular, reliable, and extensible deployment configuration suitable for scaling citizen science workflows and machine learning pipelines in a production research environment.

5.5 Design Rationale

- **Functional View**: We expanded the core modules by adding `VideoChunker` and `AnnotationAggregator` to the Data Management and Citizen Science modules, and `TrainingOrchestrator` to the Machine Learning module. These components expose new interfaces (`StorageServices`, `CitizenSciencePlatform`, `TaskScheduler`, `HPCComputeClusterInterface`) that encapsulate video split-

ting, batch annotation aggregation, and asynchronous training orchestration. This enhances cohesion within each module and decouples workflows, enabling independent evolution and scalable processing.

- **Information View**: We extended the Darwin Core–compliant SQLite schema with new tables—`Chunks`, `AnnotationSummaries`, `JobMetadata`, and `Logs`—to capture outputs from the VideoChunker, AnnotationAggregator, TrainingOrchestrator, and LoggingService. This maintains standard interoperability for biodiversity data while providing relational support for traceable linkage of video segments, aggregated annotations, training runs, and operational logs.

- **Development View**: The codebase has been refactored into fine-grained packages—`kso.chunker`, `kso.annotations`, `kso.scheduler`, and `kso.logging`—alongside microservice modules (`video-chunker-service`, `annotation-aggregator-service`, `scheduler-service`, `logging-service`, `hpc-gateway`). This modular hierarchy enables parallel workstreams for video, annotation, scheduling, and logging subsystems, reducing merge conflicts and streamlining CI/CD for each service.

- **Deployment View**: Each new component is deployed as a containerized microservice: `video-chunker` and `annotation-aggregator` on CPU nodes, `hpc-gateway` on GPU-enabled HPC nodes, and `scheduler` and `logging-service` on dedicated orchestration hosts. The Streamlit front-end and SQLite database remain on general-purpose nodes, while logs are forwarded to an ELK stack. This mapping optimizes resource usage, allows horizontal scaling of batch tasks, and ensures end-to-end observability.